

DOKUMENTACJA TECHNICZNA & BIZNESOWA

CSS Onboarding Sembot

Ścieżki rejestracji Google Comparison Shopping Service — opis kompletny

FRONTEND

proxy_sembot — Go + templ + HTMX

BACKEND

sembot-laravel — Laravel 11, DDD

PR FRONTEND

#458 — codegen/jarek-sciezkazcssnowysembot

PR BACKEND

#4058 — codegen/jarek-sciezkazcssnowysembot

TESTY E2E

29/29 ✓ wszystkie przechodzą

ŚCIEŻKI

CSS Shop + CSS Agency

Spis treści

CZĘŚĆ 1 — KONTEKST I ARCHITEKTURA

1

Czym jest CSS i dlaczego odrębna ścieżka?

Kontekst biznesowy, różnice od standardowej rejestracji

2

Przegląd architektury

Diagramy przepływu obu ścieżek, mapowanie URL → handler → endpoint

CZĘŚĆ 2 — ŚCIEŻKA CSS SHOP

3

Rejestracja — /auth/register?type=css-shop

Formularz, co zapisuje backend, konto css_shop

4

Wybór ścieżki — /auth/css

Strona po zalogowaniu, routing do Shop lub Agency

5

Formularz sklepu — /css/shop

Pola formularza, walidacja, co dzieje się po submit

6

Strona sukcesu — /css/shop/done

Potwierdzenie, informacja o aktywacji CSS, link do panelu

7

Audit Bridge — /onboarding/audit-bridge

Spinner podczas analizy strony, dalszy onboarding

CZĘŚĆ 3 — ŚCIEŻKA CSS AGENCY

8

Rejestracja — /auth/register?type=css-agency

Formularz, konto css_agency, brak auto-trialu

9

Ekran agencji — /css/agency

Checklist White Label, trial box, CTA kontaktowe

CZĘŚĆ 4 — SZCZEGÓŁY TECHNICZNE

10

Endpointy backendu

Pełna specyfikacja: URL, metody, middleware, odpowiedzi

11

Poprawki wprowadzone w tej iteracji

Opis problemów, co zmieniliśmy i dlaczego

1. Czym jest CSS i dlaczego odrębna ścieżka rejestracji?

Czym jest Google CSS?

CSS (Comparison Shopping Service) to program Google Ads dla partnerów porównujących ceny produktów. Sklepy zarejestrowane przez agencję CSS uzyskują dostęp do Google Shopping bez bezpośredniej obsługi Google — kampanie prowadzi agencja CSS w ich imieniu.

Sembot działa jako jeden z partnerów CSS, obsługując zarówno sklepy-klientów (**CSS Shop**) jak i agencje zarządzające wieloma sklepami (**CSS Agency**).

CSS Shop

- Sklep e-commerce rejestrujący się przez Sembot CSS
- Podaje dane sklepu: feed produktowy, GMC ID, domenę
- Otrzymuje 14-dniowy darmowy trial Sembota
- CS Sembota aktywuje CSS w Google w ciągu 24–48h
- Dalej przechodzi przez standardowy onboarding (audyt, połączenia)

CSS Agency

- Agencja zarządzająca kampaniami CSS dla wielu sklepów
- Konfiguruje White Label (logo, własna domena Sembota)
- Nie tworzy projektu przy rejestracji — zarządza projektami klientów
- Trial aktywowany ręcznie przez button w panelu (/css/agency)
- Kontakt z CS przez formularz lub e-mail

Dlaczego odrębna ścieżka rejestracji?

Standardowa ścieżka rejestracji Sembota zakłada, że user weryfikuje adres e-mail przed pierwszym użyciem aplikacji. CSS Shop i Agency to partnerzy potwierdzeni przez Google CSS — ich konta są **pre-weryfikowane przez partnera**, więc weryfikacja e-mail jest zbędna.

Problem: middleware verifiedEmail → 403 dla nowych użytkowników CSS

Nowi użytkownicy CSS mają `email_verified_at = NULL` w bazie danych. Endpoint `POST /api/v1/projects` jest w grupie middleware `verifiedEmail` — każdy call bez zweryfikowanego emaila zwraca **403 Access Denied**.

Rozwiązanie: dedykowany endpoint `POST /api/v1/css/shop/project` poza grupą `verifiedEmail`, który auto-weryfikuje email przy pierwszym wywołaniu (partner-verified flow).

2. Przegląd architektury

Ścieżka CSS Shop

Rejestracja

GET /auth/register?type=css-shop

Formularz z parametrem type →
account_type=css_shop w bazie



Logowanie + sesja

POST /login → cookie AES-GCM

JWT Passport + WorkspaceID zapisany w
zaszyfrowanym cookie



Wybór ścieżki

GET /auth/css

Użytkownik klika "Sklep" → redirect do /css/shop



Formularz sklepu

GET /css/shop

Pola: Nazwa, Feed URL, Domena (auto), GMC ID



Submit → Tworzenie projektu

POST /css/shop → API /css/shop/project +
/css/shop/notify

Auto-weryfikacja emaila, projekt SIMPLE, feed,
powiadomienie CS



Strona sukcesu

GET /css/shop/done

Potwierdzenie + info o aktywacji w 24–48h



Ścieżka CSS Agency

Rejestracja

GET /auth/register?type=css-agency

Formularz z parametrem type →
account_type=css_agency, brak auto-trialu



Logowanie + sesja

POST /login → cookie AES-GCM

Identycznie jak CSS Shop



Wybór ścieżki

GET /auth/css

Użytkownik klika "Agencja" → redirect do
/css/agency



Ekran agencji

GET /css/agency

Checklist White Label, trial box, CTA kontaktowe



Start trialu (opcjonalnie)

POST /api/v1/css/start-trial

+14 dni trial_ends_at na workspace

Audit Bridge (opcjonalnie)

GET /onboarding/audit-bridge

Spinner analizy strony → standardowy onboarding

Mapowanie front ↔ back

Frontend (proxy_sembot)	Backend (sembot-laravel)	Co robi
POST /css/shop	POST /api/v1/css/shop/project POST /api/v1/css/shop/notify	Tworzy projekt + powiadomienie CS
GET /css/agency	— (tylko GET, brak action)	Statyczny ekran agencji
POST /css/agency/start-trial wkrótce	POST /api/v1/css/start-trial	Aktywuje trial agencji

3. Rejestracja CSS Shop (/auth/register?type=css-shop)

Sembot

★ **CSS bezpłatny na zawsze**
Aktywacja przez nasz zespół w ciągu 24–48h.
W zestawie: 14 dni Sembota gratis —
Marketer AI pokazuje które produkty
zarabiają, które tracą i pomaga to naprawić.

Krok 2 z 3

Założ konto
Zacznij korzystać z Sembot w kilka sekund

E-mail *

Hasło *

Minimum 12 znaków, w tym małe i wielkie litery, cyfra oraz znak specjalny

Powtórz hasło *

Założ konto

Masz już konto? Zaloguj się

GET /auth/register?type=css-shop — formularz rejestracji z parametrem css-shop

Opis dla użytkownika

Sklep trafia na tę stronę przez unikalny link partnerski CSS (np. `css.sembot.com/register?type=css-shop`). Formularz wygląda identycznie jak standardowa rejestracja, ale parametr `?type=css-shop` w URL zmienia typ tworzonego konta.

Co dzieje się w tle (front)

Handler `Auth.ShowSignup` w `internal/handlers/auth.go` czyta parametr `?type=` i przekazuje go przez ukryte pole formularza. Po submit — `Auth.DoSignup` wysyła go do backendu.

Co dzieje się w tle (backend)

```
// sembot-laravel: app/Http/Controllers/Api/Auth/LoginController.php
// str_replace normalizuje css-shop → css_shop (dash → underscore)
$accountType = AccountType::tryFrom(str_replace('-', '_', $request->account_type));
// Zapisuje jako account_type = 'css_shop' w tabeli users (landlord DB)

// app/Types/AccountType.php
case CSS_SHOP = 'css_shop'; // hasAccessToFreeSembot() → true (darmowy dostęp)
case CSS_AGENCY = 'css_agency';
```

Co jest zapisywane w bazie po rejestracji

- Tabela `users` (landlord DB): `account_type = 'css_shop'`
- `email_verified_at = NULL` — celowo, weryfikacja odbywa się przy tworzeniu projektu
- Workspace (account) tworzony automatycznie przez event `Registered`
- Wysyłany jest `CSSWelcomeMail` zamiast standardowego welcome maila
- Brak auto-trialu przy rejestracji (trial nadawany po aktywacji CSS przez CS)

8. Rejestracja CSS Agency (/auth/register?type=css-agency)

Sembot

★ **White Label CSS — 1 rok bezpłatnie**
Własna marka w Google Shopping. W zestawie: 14 dni Sembota gratis — Marketer AI zarządza kampaniami wszystkich klientów z jednego miejsca.

Krok 2 z 3

Założ konto
Zacznij korzystać z Sembot w kilka sekund

E-mail *

Hasło *

Minimum 12 znaków, w tym małe i wielkie litery, cyfra oraz znak specjalny

Powtórz hasło *

Założ konto

Masz już konto? Zaloguj się

GET /auth/register?type=css-agency — formularz rejestracji z parametrem css-agency

Opis dla użytkownika

Agencja trafia na tę stronę przez osobny link partnerski (`?type=css-agency`). Formularz jest identyczny wizualnie jak CSS Shop, ale parametr zmienia typ konta i zachowanie systemu.

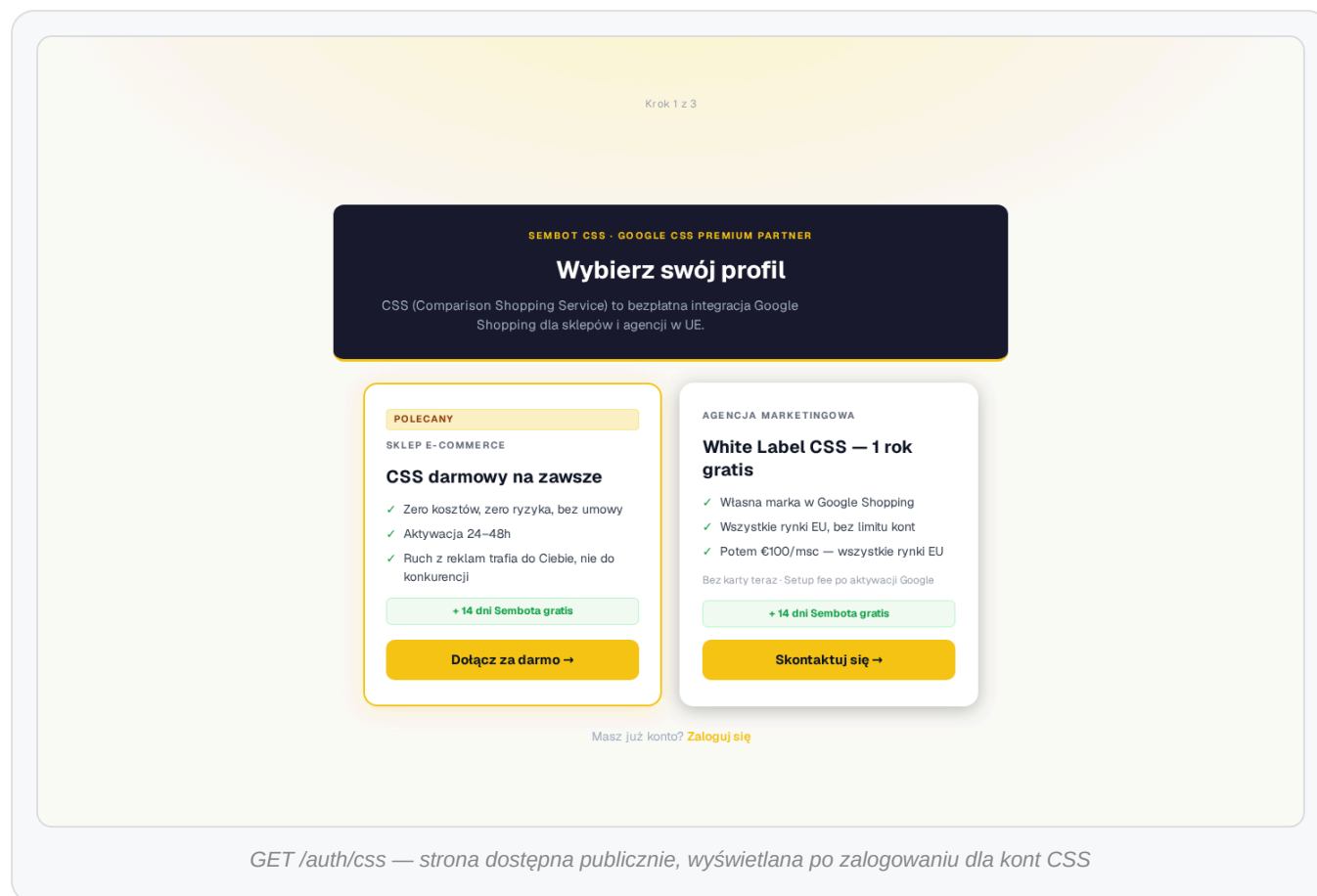
Różnice od CSS Shop

Aspekt	CSS Shop	CSS Agency
account_type	css_shop	css_agency
Auto-trial przy rejestracji	Nie (nadawany przez CS po aktywacji)	Nie — skip w UserRepository
Projekt po rejestracji	Tworzony w /css/shop	Nie tworzony — agencja zarządza projektami klientów
Email powitalny	CSSWelcomeMail (wariant shop)	CSSWelcomeMail (wariant agency)
Dalszy onboarding	/auth/css → /css/shop → projekt	/auth/css → /css/agency → White Label

Techniczny detal: skip auto-trialu

W `app/Repositories/UserRepository.php` dodano warunek: jeśli `account_type === 'css_agency'`, pomiń nadawanie domyślnego trialu przy rejestracji. Trial agencji jest aktywowany ręcznie przez endpoint `POST /api/v1/css/start-trial` po tym, jak agencja kliknie odpowiedni button w panelu.

4. Wybór ścieżki CSS (/auth/css)



Opis dla użytkownika

Po zarejestrowaniu i zalogowaniu użytkownik CSS trafia na tę stronę. Widzi dwa przyciski: **Sklep** (dla właścicieli sklepów e-commerce) i **Agencja** (dla firm zarządzających kampaniami CSS wielu klientów). Wybór określa dalszy przepływ.

Opcja: Sklep →

Redirect do `/css/shop`. Użytkownik będzie wypełniał dane swojego sklepu i feed produktowy.

Konto: `css_shop`

Opcja: Agencja →

Redirect do `/css/agency`. Użytkownik zobaczy checklistę White Label i informacje o trialu.

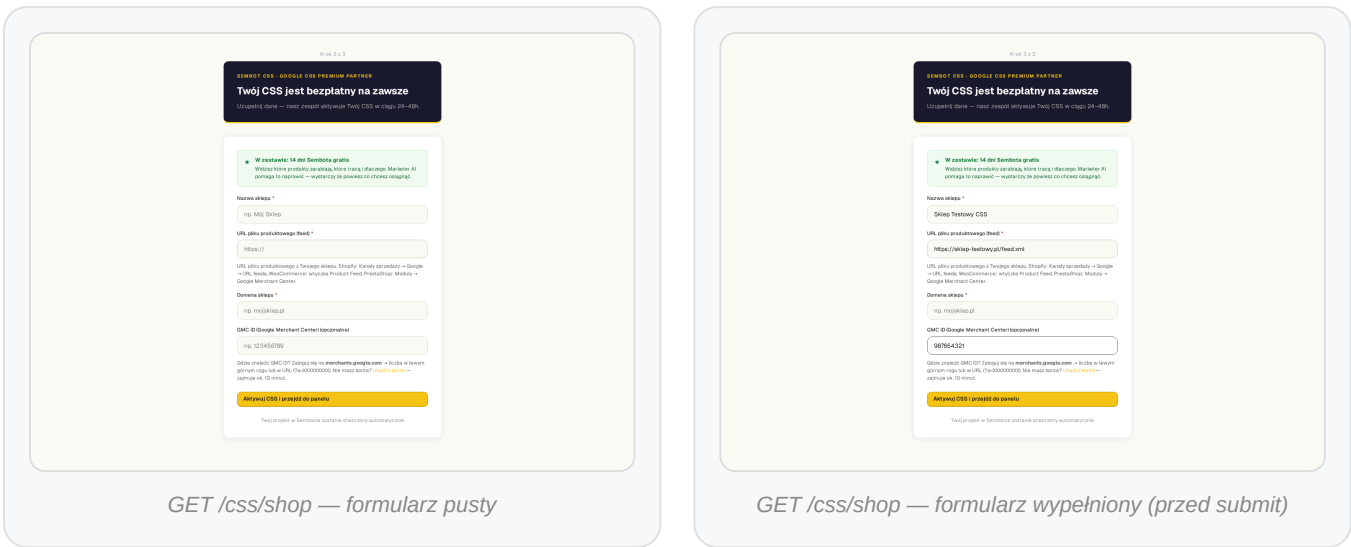
Konto: `css_agency`

Szczegóły techniczne

Strona jest **dostępna publicznie** (nie wymaga sesji — handler `Auth.ShowCSSPath` nie woła `requireSession`). Renderuje statyczny templ `pages.CSSPath()`. Linki "Sklep" i "Agencja" to zwykłe `<a href>` wskazujące na `/css/shop` i `/css/agency` (te już wymagają sesji).

Jeśli backend zwróci nieprawidłowy `account_type` po rejestracji (np. z powodu błędu normalizacji `dash` → `underscore`), user mógłby trafić na stronę `/auth/css` ale po wyborze "Sklep" zobaczyć błąd — bo middleware konta sprawdza `account_type`. Warto dodać guard na tej stronie weryfikujący że zalogowany user ma `account_type` `css_shop` lub `css_agency`.

5. Formularz sklepu (/css/shop)



Pola formularza

Pole	Wymagane	Opis
Nazwa sklepu (<code>shop_name</code>)	✓ Tak	Nazwa pojawi się jako nazwa projektu w Semboocie i w emailu do CS
URL feeda produktowego (<code>feed_url</code>)	✓ Tak	Google Shopping / XML feed. Walidowany jako URL. Po wpisaniu automatycznie uzupełnia pole Domena (Alpine.js: <code>new URL(feedUrl).hostname</code>)
Domena sklepu (<code>domain</code>)	✓ Tak	Auto-uzupełniana z feed URL, edytowalna. Służy jako domain projektu w Semboocie
GMC ID (<code>gmc_id</code>)	Nie	Google Merchant Center ID. Przekazywany do CS w emailu notyfikacyjnym. Jeśli nie podany — CS może uzupełnić ręcznie

Zachowanie Alpine.js (UX)

- Zmienne Alpine: `feedUrl` , `domain` , `submitting`
- Auto-uzupełnienie domeny: `x-on:input` na polu feed URL → `new URL(feedUrl).hostname.replace(/^www\./, '')`
- Blokada podwójnego submit: `x-on:submit="submitting = true"` na `<form>` (nie na buttonie — eliminuje race condition Alpine.js)
- Button disabled podczas wysyłania: `x-bind:disabled="submitting"`

Walidacja po stronie frontu

Handler `SubmitShop` sprawdza czy `shop_name`, `feed_url` i `domain` są niepuste. Jeśli brakuje któregoś — renderuje formularz ponownie z komunikatem błędu i prefill'em wpisanych wartości.

5b. Co dzieje się po kliknięciu "Dalej" w formularzu sklepu

1. Walidacja po stronie Go

Sprawdza że shop_name, feed_url, domain są niepuste. Błąd → re-render formularza z komunikatem.



2. POST /api/v1/css/shop/project (Laravel)

Tworzy projekt typu SIMPLE w workspace użytkownika. Auto-weryfikuje email jeśli email_verified_at=NULL (partner-verified flow). Zwraca JSON z ID i slug projektu.



3. POST /api/v1/feeds (Laravel)

Tworzy input feed z URL podanym przez użytkownika, domyślnym mapowaniem Google Shopping. Błąd feedu nie blokuje — projekt już istnieje, CS doda feed ręcznie.



4. POST /api/v1/css/shop/notify (Laravel)

Wysła email na css@sembot.com z danymi: nazwa, GMC ID, feed URL, domena, link do projektu. Ustawia in_css_program=1 na projekcie.



5. Redirect → /css/shop/done

Przekazuje w query string: ws (workspace slug), proj (project slug), name (nazwa sklepu) — do wyświetlenia na stronie sukcesu.

Kod backendu — kluczowe fragmenty

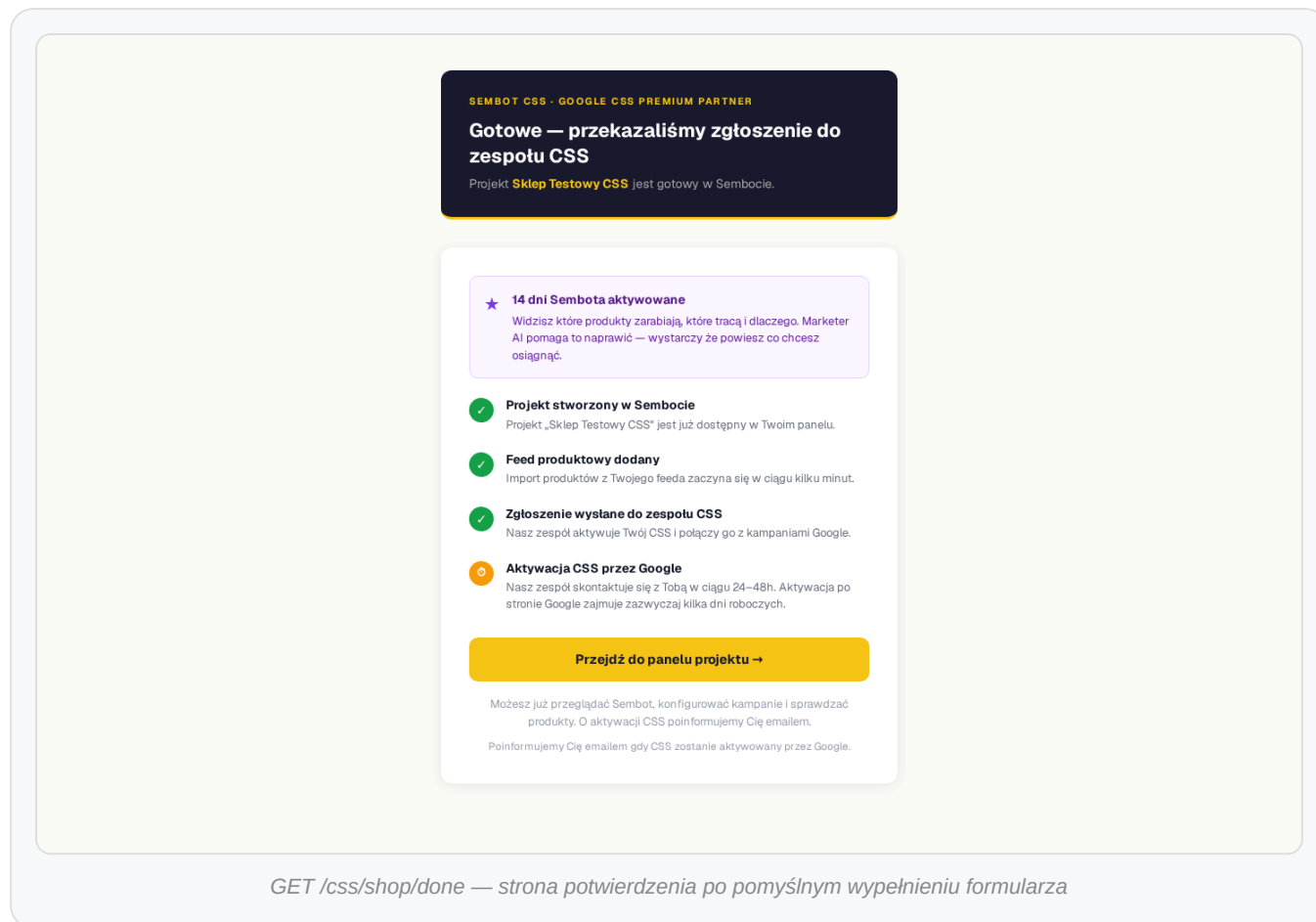
```
// CssOnboardingController::createShopProject (sembot-laravel)
// Auto-weryfikacja emaila (partner-verified flow):
if (!$user->hasVerifiedEmail()) {
    $user->markEmailAsVerified();
}
// Tworzenie projektu przez CommandBus (StoreProject):
dispatch_sync(new StoreProject(
    $uuid->string, $request->validated('name'), $request->validated('domain'),
    null, null, null, null, $user->id, $currentWorkspace->id,
    null, null, Project::SIMPLE, false, 'UTC', null,
));
// Zwraca: { "data": { "id": 123, "slug": "sklep-testowy-css", ... } }
```

Endpoint `/css/shop/project` vs standardowe `/projects`

`POST /api/v1/projects` — jest w grupie middleware `verifiedEmail` → 403 gdy email niezweryfikowany.

`POST /api/v1/css/shop/project` — poza grupą `verifiedEmail`. Przed tworzeniem projektu auto-weryfikuje email. Identyczny efekt co `/projects`, ale dostępny dla wszystkich nowych CSS userów.

6. Strona sukcesu (/css/shop/done)



Opis dla użytkownika

Użytkownik widzi tę stronę bezpośrednio po poprawnym wypełnieniu formularza sklepu. Strona potwierdza że sklep został zarejestrowany i informuje o dalszych krokach: aktywacja CSS przez Sembota trwa 24–48 godzin roboczych.

Co zawiera strona

- Potwierdzenie z nazwą sklepu (z query param `?name=`)
- Informacja o czasie aktywacji CSS (24–48h)
- Link do panelu projektu (`/p/{workspace}/{project}/tasks`)
- Informacja że CS skontaktuje się w razie pytań

Szczegóły techniczne

Handler `ShowShopDone` w `css_onboarding.go` czyta parametry z query string (`ws` , `proj` , `name`) i renderuje templ `CSSShopSuccess` . Sesja jest weryfikowana (redirect do login jeśli wygasła).

```
// URL po redirect z SubmitShop:
/css/shop/done?ws={workspace_slug}&proj={project_slug}&name={shop_name}
```

// Linki na stronie sukcesu:

href="/p/{ws}/{proj}/tasks" // → panel projektu w Sembocie

7. Audit Bridge (/onboarding/audit-bridge)



GET /onboarding/audit-bridge — spinner podczas analizy strony sklepu

Opis dla użytkownika

Po stronie sukcesu (lub bezpośrednio przez link) użytkownik trafia na tę stronę. Widzi animację ładowania i informację że Sembot analizuje jego stronę. Analiza obejmuje wykrycie czy strona to sklep, jakie produkty sprzedaje, jakiego używa CMS.

Co dzieje się technicznie

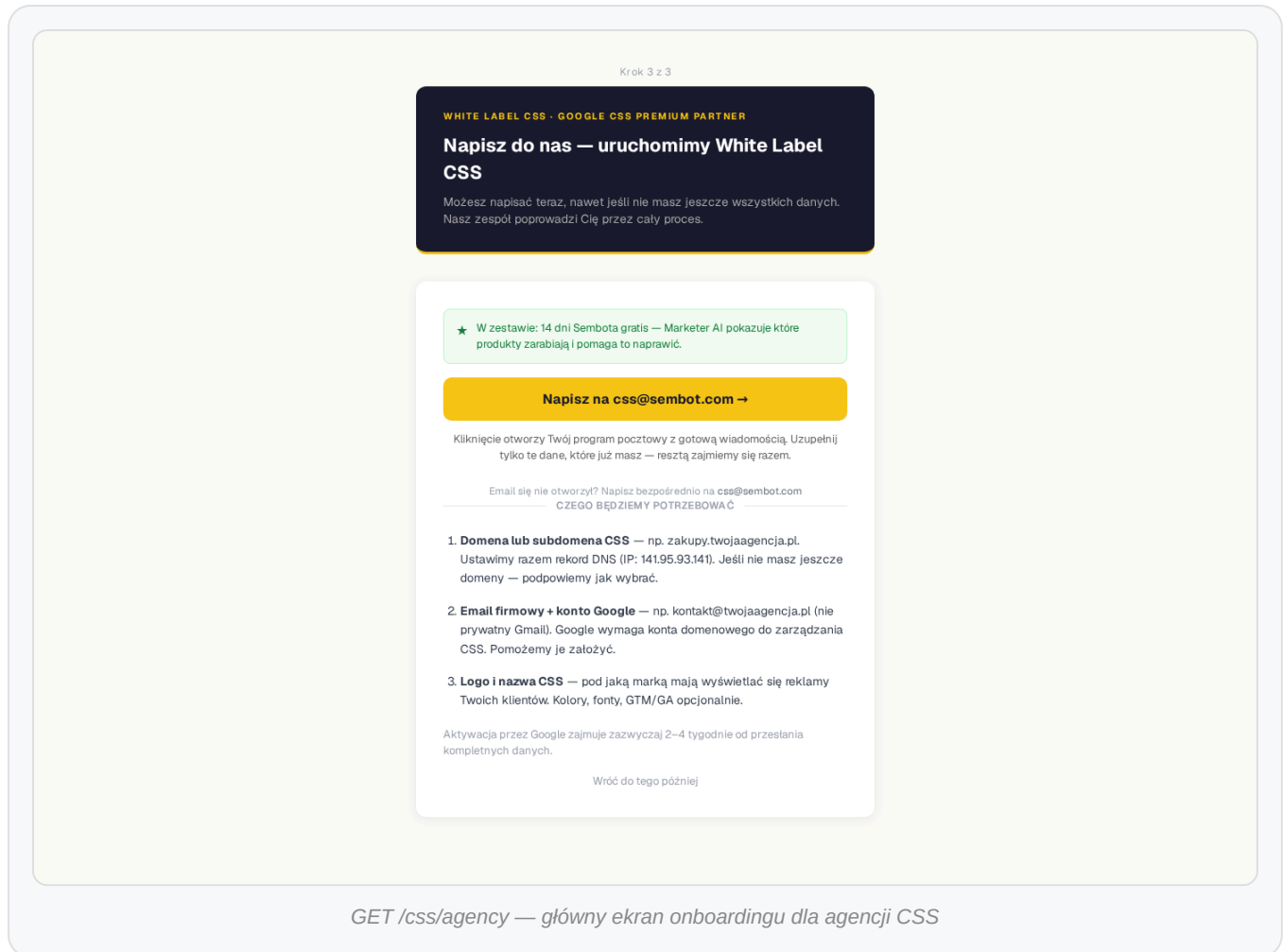
Strona co 500ms odpytuje endpoint `GET /onboarding/audit-bridge/status` (long-polling przez HTMX). Gdy audit zakończy się:

- Status `done` → redirect do `/onboarding/connections` (standardowy onboarding)
- Status `error` → komunikat błędu, możliwość przejścia ręcznie
- Timeout 5 min → redirect do connections z pominięciem audytu

Kontekst: standardowy onboarding po CSS Shop

CSS Shop po przejściu przez swój formularz trafia do **identycznego onboarding**u co standardowe konta: audit → connections (Google Ads, GMC) → CS call. Jedyna różnica to że projekt i feed były już stworzone w kroku /css/shop.

9. Ekran agencji (/css/agency)



Opis dla użytkownika

Agencja widzi tę stronę po wybraniu "Agencja" na /auth/css. Ekran zawiera trzy sekcje:

White Label — checklist

- Konfiguracja własnego logo agencji
- Ustawienie własnej domeny Sembota dla klientów
- Personalizacja palety kolorów

Konfiguracja w ustawieniach workspace → Whitelabel

Trial 14 dni

- Informacja o darmowym dostępie przez 14 dni
- Button "Zacznij trial" → wywołuje `POST /api/v1/css/start-trial`
- Ustawia `trial_ends_at = now + 14 dni` na workspace

Endpoint start-trial (backend)

```
// CssOnboardingController::startTrial (sembot-laravel)
// POST /api/v1/css/start-trial
```

```
$workspace = Workspace::find($user->last_account);  
$workspace->trial_ends_at = now()->addDays((int) config('app.trialLength', 14));  
$workspace->save();  
// Zwraca: 204 No Content
```

Stan implementacji: start-trial w proxy_sembot

Backend endpoint `POST /api/v1/css/start-trial` jest gotowy. Handler frontendu (`POST /css/agency/start-trial`) i API client (`StartCssTrial`) są zaplanowane ale jeszcze nie zaimplementowane w proxy_sembot. Button na ekranie agencji póki co może przekierowywać do kontaktu z CS.

10. Endpointy backendu (sembot-laravel)

POST /api/v1/css/shop/project

Element	Wartość
URL	<code>POST /api/v1/css/shop/project</code>
Auth	Bearer JWT (Passport), header <code>account: {workspace_id}</code>
Middleware	<code>auth:api</code> , <code>client.version</code> — NIE <code>verifiedEmail</code>
Request body	<code>name</code> (required, min:3), <code>domain</code> (required, min:3)
Response 201	<code>{ "data": { "id": 123, "name": "...", "slug": "...", "domain": "..." } }</code>
Efekty uboczne	Auto-weryfikuje email, tworzy projekt SIMPLE przez CommandBus (<code>StoreProject</code>), ustawia <code>in_css_program=1</code> pośrednio przez notify

POST /api/v1/css/shop/notify

Element	Wartość
URL	<code>POST /api/v1/css/shop/notify</code>
Auth	Bearer JWT (Passport), header <code>account: {workspace_id}</code>
Middleware	<code>auth:api</code> , <code>client.version</code> , <code>verifiedEmail</code>
Request body	<code>shop_name</code> , <code>gmc_id</code> , <code>feed_url</code> , <code>domain</code> , <code>project_url</code> (optional), <code>project_id</code> (optional)
Response 204	No Content
Efekty uboczne	Wysyła <code>CssShopNotifyMail</code> na <code>config('mail.emails.css')</code> , ustawia <code>project.in_css_program = true</code> jeśli podano <code>project_id</code>

POST /api/v1/css/start-trial

Element	Wartość
URL	<code>POST /api/v1/css/start-trial</code>

Auth	Bearer JWT (Passport)
Middleware	<code>auth:api</code> , <code>client.version</code>
Request body	Brak — workspace brany z sesji (<code>\$user->last_account</code>)
Response 204	No Content
Efekty uboczne	Ustawia <code>workspace.trial_ends_at = now() + 14 dni</code> , idempotentne (nadpisuje istniejący trial)

11. Poprawki wprowadzone w tej iteracji

Poprawka 1: 403 Access Denied dla nowych użytkowników CSS Shop

Problem: Nowi użytkownicy CSS Shop mają `email_verified_at = NULL`. Endpoint `POST /api/v1/projects` jest w grupie middleware `verifiedEmail` — zwracał 403 dla każdego nowego rejestracji.

Rozwiązanie:

- Nowy endpoint `POST /api/v1/css/shop/project` poza grupą `verifiedEmail`
- Auto-weryfikacja email przy wywołaniu: `$user->markEmailAsVerified()`
- Go frontend: zmieniono `h.API.CreateProject()` → `h.API.CSSCreateProject()` w `SubmitShop`
- Nowa metoda `CSSCreateProject` w `internal/api/projects.go`

Poprawka 2: Race condition Alpine.js przy submit formularza

Problem: `x-on:click="submitting = true"` na buttonie + `x-bind:disabled="submitting"`. Alpine microtask wyłączał button przed tym jak przeglądarka zdążyła wysłać formularz → submit był blokowany.

Rozwiązanie: Przeniesiono `x-on:submit="submitting = true"` na element `<form>`. Event `submit` na formularzu odpala się po wysłaniu żądania, nie przed.

Poprawka 3: Brakujący route GET /css/shop/done

Problem: Handler `ShowShopDone` istniał w `css_onboarding.go`, ale brakowało rejestracji route'u w `cmd/server/main.go`. Po submit formularza redirect trafiał w 404.

Rozwiązanie: Dodano `authedUnscoped("GET", "/css/shop/done", cssOnbH.ShowShopDone)` w `main.go`.

12. Checklist dla zespołu produkcyjnego

Przed wdrożeniem na produkcję sprawdź poniższe punkty:

Backend (sembot-laravel)

Co sprawdzić	Uwagi
✓ Route <code>POST /css/shop/project</code> jest poza grupą <code>verifiedEmail</code>	Sprawdź <code>routes/v1.php</code> sekcja ~590
✓ Route <code>POST /css/shop/notify</code> istnieje	W tej samej sekcji routes
✓ Route <code>POST /css/start-trial</code> istnieje	W tej samej sekcji routes
✓ <code>config('mail.emails.css')</code> ustawiony w <code>.env</code>	Adres na który trafia powiadomienie o sklepie (<code>css@sembot.com</code>)
✓ <code>CSSWelcomeMail</code> ma poprawne szablony blade	Sprawdź <code>resources/views/emails/css-*</code>
✓ <code>app.trialLength</code> ustawiony w config (default: 14)	Liczba dni trialu agencji
✓ Migracje: kolumna <code>in_css_program</code> w tabeli <code>projects</code>	Powinna istnieć z poprzednich migracji
✓ Typ konta <code>css_shop</code> i <code>css_agency</code> w enum <code>AccountType</code>	Zweryfikuj czy migracja enum jest na prod DB

Frontend (proxy_sembot)

Co sprawdzić	Uwagi
✓ Route <code>GET /css/shop/done</code> zarejestrowany w <code>main.go</code>	Dodany w tej iteracji — sprawdź czy jest w buildie prod
✓ <code>CSSCreateProject</code> woła <code>/css/shop/project</code> a nie <code>/projects</code>	Sprawdź <code>internal/api/projects.go</code>
✓ Templ pliki wygenerowane przed buildem (<code>templ generate ./...</code>)	CI powinno to robić automatycznie
✓ i18n klucze dla ścieżki CSS istnieją w <code>locales/pl/</code> i <code>locales/en/</code>	Klucze <code>css.*</code>

Do implementacji (następna iteracja)

Task	Gdzie
Handler <code>POST /css/agency/start-trial</code> w proxy_sembot	<code>internal/handlers/css_onboarding.go</code>
API client <code>StartCssTrial</code> wywołujący <code>POST /api/v1/css/start-trial</code>	<code>internal/api/</code>
Button "Zaczynj trial" na ekranie <code>/css/agency</code> podpięty do handlera	<code>pages/css_onboarding_agency.templ</code>
Guard na <code>/auth/css</code> weryfikujący <code>account_type</code> (<code>css_shop/css_agency</code>)	<code>internal/handlers/auth.go</code>

Status testów E2E

Wszystkie 29 testów CSS e2e przechodzi. Testowane scenariusze obejmują: rejestrację, submit formularza sklepu, redirect po submit, tworzenie projektu w DB, ustawienie `in_css_program=1`.

Repo testów: `sembot-test-automation` — plik `css-e2e-test.js`